ESP-FC Next

Dual-chip ESP32-P4 + ESP32-C5 flight controller and firmware for indoor autonomous swarm research

Item	Value
Document	Project plan, gap analysis and Phase 1 task list
Version / date	v1.7 - October 28, 2026
Base project	Espressif ESP-Drone (GPL-3.0), master after PR #137 (ESP-IDF 5.5 / 6.1)
First target	Indoor swarm of 4 drones, USA
Guiding rule	Nothing is ordered or assembled until math, simulation and code/PCB review gates pass

Prices and timelines in this document are rough estimates for planning only and should be re-checked with suppliers before purchasing.

1. Executive summary

The goal is a cheap but highly accurate flight controller (FC) and firmware built on Espressif chips, validated in simulation before any hardware is assembled. The first application is an indoor swarm of four 2-inch ducted drones for autonomy research; a surveillance variant (camera, video streaming, GPS) follows on the same board.

- **Two chips:** ESP32-P4 runs flight control, state estimation and (later) vision; an FCC-certified ESP32-C5 module handles all radio work (Wi-Fi 6 dual-band, ESP-NOW swarm links).
- **Accurate, low-cost sensors:** ICM-42688-P IMU on SPI at up to 8 kHz, BMP581 barometer, optical flow + time-of-flight, INA226 power monitoring, bidirectional DShot RPM telemetry.
- **Shared indoor positioning:** low-cost motion capture (PS3 Eye cameras + IR markers) gives all four drones one coordinate frame and ground-truth for measuring accuracy; UWB later.
- **Simulation-first:** six verification gates (G1-G6) covering math models, software-in-the-loop, static analysis/fuzzing/AI review, PCB checks, hardware-in-the-loop, then real flight.
- **Compatibility:** keeps the Crazyflie CRTP protocol so Crazyswarm2 (ROS 2) and its simulator can be used directly for swarm research.

Confirmed decisions

Topic	Decision
Use case	Indoor swarm research first (4 drones); surveillance variant later
Location / rules	USA - indoor flight first; FCC-certified radio module
Scope	New FC PCB + firmware, both open (GPL-3.0 accepted)
Chips	ESP32-P4 (flight + vision) + ESP32-C5 module (radio)
Frame	2-inch (85-90 mm) ducted brushless whoop, off-the-shelf first, custom 3D-printed later
Motors / ESC	1103-1104 brushless, separate 4-in-1 ESC with Bluejay (bidirectional DShot)
Positioning	Low-cost mocap first + onboard flow/ToF; UWB later
Simulator	Own validated 6-DOF model + in-process SITL first; pybullet for obstacles/cameras; Gazebo + CrazySim / Crazyswarm2 (ROS 2) for swarm work
Ground control	Laptop + gamepad over Wi-Fi to start; ESP-NOW USB swarm dongle; ELRS safety radio for every real flight
P4 <-> C5 link	Custom SPI protocol (recommended over ESP-Hosted for ESP-NOW control and timing)

Prior art to benchmark

rtlopez/esp-fc (Betaflight-configurator-compatible ESP32 firmware), ArduPilot's experimental ESP32 port, Crazyflie upstream, and the open-source Low-Cost-Mocap project (ESP32 drone swarm with PS3 Eye cameras).

Progress so far (v1.7)

Task	Status	Key result
1.1 Repo + toolchain	Done	Private GitHub repo, Docker ESP-IDF 5.5 (Colima), CI builds P4 + S3 + C5 firmware
1.2 Portable flight core	Done	18 Crazyflie algorithm files build on the PC unchanged (fc_core); HAL = existing driver interfaces
1.3 Unit tests	Done	Started with 7 suites, 45 tests; now 14 suites, 132 tests under ASan/UBSan. First pass found and fixed 2 real bugs: Kalman 0/0 NaN when gyro reads exactly zero (could reset the FC in flight) and a 64-bit invSqrt over-read
1.4 Gate G1 math model	Done - PASS	Stock ESP-Drone timing (500 Hz) + stock gains: roll/pitch rate loop PM 24.6 deg, GM 4.5 dB (fails). Planned 4 kHz loop: 3.5x worst-case bandwidth (11.3 vs 3.3 Hz) at equal margins; robust gains pass 1000-sample Monte Carlo (+/-20 % mass, thrust, inertia); interactive 3D replay viewer
1.5 SITL	Done - G2 hover PASS	Real C flight code (fc_step + synchronous Kalman) flies the 6-DOF model in 1 ms ctypes lockstep; 84 params / 103 logs by cfclient names; steady hover error < 3 cm, estimator error < 1 cm. Findings: posCtlPid.zKi causes 11.4 cm takeoff overshoot (0.2 cm with zKi = 0); no velocity feedforward in the 2021 position controller (0.3-0.4 s tracking lag); stock link-loss behaviour drops the drone. Gym-style RL env.
1.6a Flight-core simulation	Simulation checks pass; hardware pending	1/4 kHz host variants, 500 Hz EKF at 4 kHz, G1-generated board profiles, velocity/acceleration feedforward, exact sqrt on FPU builds, arming/tumble guards. Camera-tracked figure-8: about 4-5 cm maximum error in nominal simulation (10 cm target); flow-only absolute positioning FAILS (FLOW-01).
1.6b Motor vibration + RPM notch	Simulation checks pass; hardware pending	Synthetic motor-vibration model (ASSUMED amplitudes); 4-motor 1x/2x RBJ notch bank, 100 Hz RPM telemetry, 100 ms bypass. 12 deg/s assumed artifact: 3.8 -> 0.20 deg/s RMS; motor activity >250 Hz 3.1% -> 1.9% (baseline 1.9%). Hover/figure-8 tracking unchanged (LPF already attenuates the band). 500 Hz notch floor; worst sampled RPM/32-corner margin 46.2 deg / 6.9 dB (hover only: 47.5 deg / 7.2 dB).
Safety: hold + landing	Opt-in SITL; hardware pending	Command loss: fixed-pose hold then descent; camera loss: level descent with independent local ToF. LAND stays latched through touchdown, even if commands return; terminal states block automatic rearm. Nominal 1/4 kHz SITL: max descent 0.263 m/s, sampled touchdown approach 0.138-0.153 m/s; motors off at ground. Camera lost above the 1.3 m ToF range: bounded blind descent (max(6 s, entry height / 0.2 m/s)) with thrust held at 0.97-0.99 x learned hover, so a +/-0.02 g unseen accel bias still lands from 2-4 m (it climbed to 4 m and fell before; a fixed 6 s budget cut the motors at 1.9 m after loss at 3 m). Permanent loss of every height source still ends in FAULT (from 1 m: 0.58 m/s contact, then motor stop on the ground at 6 s; before: 4.2 m/s fall), not safe landing.
Local flow braking	SITL checks pass; hardware pending	Healthy flow/range enables zero-velocity braking after camera loss instead of passive coasting. Nominal excursion 1.71-1.72 m -> 0.277-0.290 m; sampled stress worst 0.349 m at 1/4 kHz. Stale flow degrades to level descent; restored jumped camera frames are not reacquired. No absolute-positioning or total-height-loss solution claimed.
Input plausibility	Done (host/SITL)	Audit found one NaN IMU sample poisoned the EKF permanently and tripped asserts (motor stop on target) in both modes. Bad IMU samples are now held for up to 5 ms, then disarm/FAULT; non-finite position/ToF/flow are dropped in legacy mode too. Regression tests failed before the fix.
1.7 Link protocols	In progress	docs/protocols.md: 64-byte P4-C5 SPI frame with CRC-16, 20-byte 50 Hz swarm status, 32-message anti-replay window; unit tests + libFuzzer (make link-fuzz, 1 h run: 4.1 B inputs, no crash); builds on the C5 toolchain. AES-128-CCM envelope built on IDF mbedtls 3.6.7 with RFC 3610 vector tests; C5 boot self-check not yet run on a board. C5 bridge (fc_radio): envelope <-> SPI frame, ground-only commands, NVS-reserved counters (no nonce reuse across reboot); C5 image wires ESP-NOW + SPI slave. P4 side: CONFIG_TRANSPORT_C5_SPI CRTP link (500 Hz SPI master, make flight-c5link, CI). Both ends host-tested and compile-checked; not run on a board. CRTP is addressed (drone id or all, inside the ciphertext): a ground command no longer reaches every drone. Ground C5 (id 0) streams the same frames to the PC over USB serial (tools/espfc_link.py, cross-checked against the firmware codec). Provisioning: tools/provision_radio.py. cflib driver tools/espfc_cflib.py (espfc://<id>@<port>); Crazyswarm2 needs a 2-line register() patch, untried.

Task	Status	Key result
fc_step on target	Builds; never flown	CONFIG_ESPFC_FC_STEP runs fc_step in ESP-Drone's stabilizer task (make flight-fcstep, CI on ESP32-S3: +7 KB). Mocap/pose/ToF/flow reach it through a static queue; only new commander packets (and high-level commander points while the link is alive) feed its watchdog; host tests cover the glue; the driver LPF is compiled out so IMU samples are filtered once. P4 image is still the diagnostic profile (no board yet).
1.8 Swarm simulation	SITL checks pass	make swarm: four firmware copies in lockstep, altitude layers 1.0-2.05 m, square formation and diagonal swap. Nominal, link loss, camera dropout and low battery (in formation and mid-swap, top layer) all land safely; minimum separation 0.35 m. No inter-drone downwash; low battery is a voltage step, no discharge model.
1.9 G3 quality gate	Met on host	cppcheck and clang-tidy clean (make core-lint / core-tidy, CI); -Wall -Wextra on fc_core sources; line coverage 80.8% of fc_step-reachable code (INDI/Mellinger excluded, planner not built on host); review checklist (docs/review_checklist.md). Timing budget: fcTime logs on target, unmeasured until a board runs.

4 kHz is a host/SITL configuration, not a measured ESP32-P4 runtime. The default images still use the existing ESP-Drone task loop; fc_step on target is opt-in and compile-checked only. Driver integration, CPU deadlines, Wi-Fi/camera contention and hardware flight are unvalidated. Dynamic-notch/RPM filtering was added in Task 1.6b as a simulation-only model.

Task 1.6b is a synthetic stress model, not measured vibration. It adds a gyro artifact only, not frame resonance or body torque. The notch reduces high-frequency gyro and motor-command activity but did not improve position tracking in simulation; it does not fix FLOW-01 (flow-only absolute XY drift). Review found the initial 150 Hz notch floor failed off-hover margins; it is now 500 Hz per section. Q=5 sampled linear checks are not an all-speed or hardware guarantee. RPM calibration, pole-pair conversion, ESC telemetry latency and real vibration spectra require hardware blackbox data. The 4 kHz runtime remains host/SITL only; the production P4 image still uses the existing ESP-Drone task loop.

Latched landing is opt-in in the portable runtime, not activated on hardware. Thresholds and touchdown dwell are engineering assumptions. Without local flow, moving camera-loss tests drift about 1.7-2.0 m. Flow/range braking reduces the tested excursion but does not provide collision-safe swarm flight; all-height loss can still cause a fall. Finite frozen/replayed sensor data is not detected. The four-drone fault/landing gate passes in SITL only (no downwash, battery or radio timing). BMP581 pressure noise is not equivalent to altitude accuracy: barometer/contact redundancy is still research, not an implemented safe-touchdown fallback.

2. ESP-Drone gap analysis

Reviewed: the ESP-Drone documentation (Get Started, Hardware Reference, Drivers, Flight Control System, Communications) and the GitHub repository. Note: upstream merged a large refactor on Oct 26, 2026 (PR #137) adding ESP-IDF 5.5/6.1, ESP32-P4/S31 targets, disarmed-by-default startup, NVS configuration storage and malformed-packet rejection. Items partly fixed by it are marked *(partly fixed upstream)*.

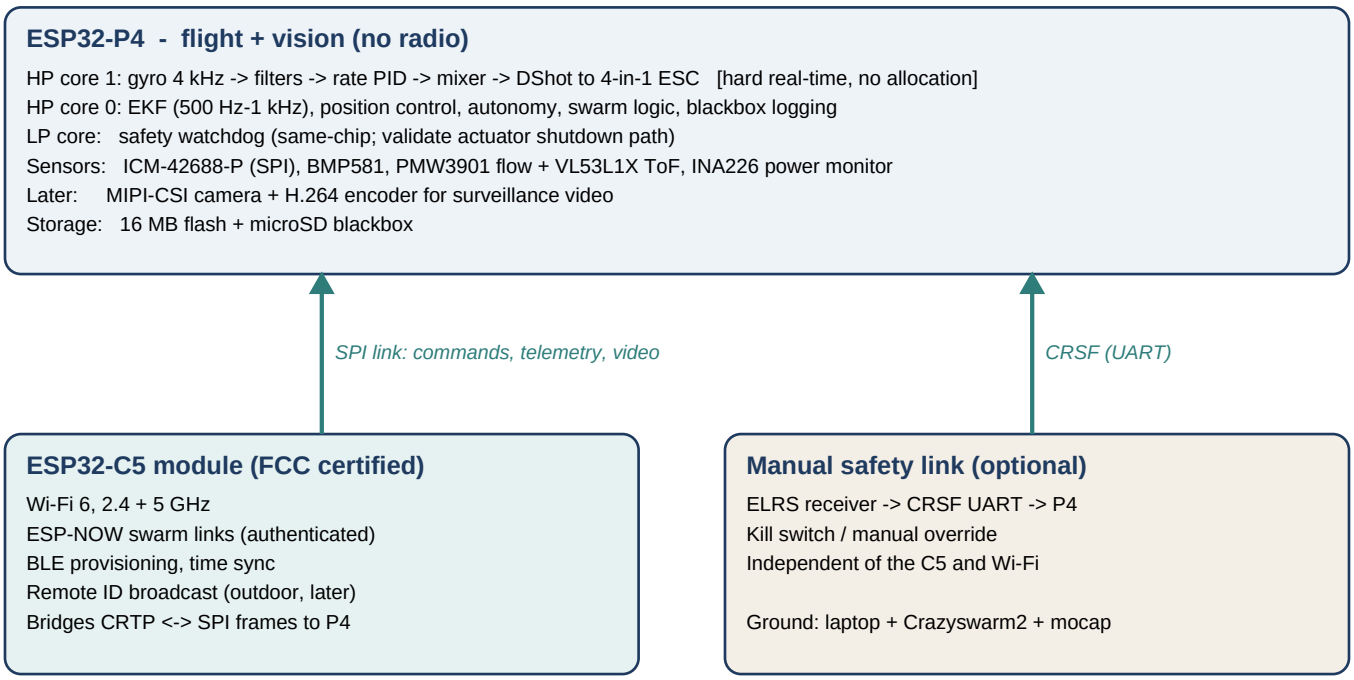
2.1 Hardware

Area	Current state	Gap	Opportunity
Main chip	ESP32-S2 (single core) on reference board	Kalman estimator must run at low priority or the watchdog fires	Dual-chip P4 + C5: dedicated real-time core, radio isolated
IMU	MPU6050 on I2C, as low as 125 Hz sampling	Old, noisy, slow bus, single unit	ICM-42688-P on SPI, 4-8 kHz, data-ready interrupt, FIFO; optional 2nd IMU
Baro / compass	MS5611, HMC5883 (discontinued)	Baro altitude hold reported broken in open issues	BMP581 barometer; LIS3MDL compass (optional)
Motors	Brushed 716/720 only, simple PWM	No brushless/ESC support, no RPM telemetry	DShot300/600 via RMT, bidirectional DShot RPM
Power	1S, voltage via VBAT/2 divider	No current sensing; brownout issues reported	INA226, separate logic/motor rails, bulk caps, reverse-polarity protection
Logging	None onboard	No post-flight vibration/tuning analysis	Blackbox to flash / microSD
Positioning	Flow + ToF, indoor only	No GPS (issue open since 2022); no shared swarm frame	Mocap + flow; UWB; M10 GPS later
RC input	Wi-Fi app / gamepad only	No standard RC receivers	CRSF / ELRS over UART
Openness	Non-KiCad design files	KiCad export requests; S3/V2 schematics missing	Fully open KiCad design

2.2 Firmware

Area	Gap	Opportunity
Codebase age	Flight core is Crazyflie 2021.01 (about 5 years old)	Rebase on newer Crazyflie features: arming supervisor, controllers, Kalman fixes
Loop timing	I2C IMU; flight loop competes with Wi-Fi tasks	Flight loop pinned to its own core at 2-4 kHz with measured jitter
Filtering	Only a basic 2-pole low-pass	Dynamic notch (FFT), RPM notch, D-term LPF, feedforward, airmode
Control	Only PID tuned; Mellinger/INDI untested	Tuned INDI; acro/rate modes
Failsafe	Basic link-timeout cutoff (disarmed by default: partly fixed upstream)	Auto-land on link loss / low battery, crash detection, geofence, preflight checks
Security	Fixed AP password 12345678; no command authentication; 1-byte checksum; no replay protection	Per-device credentials, WPA3, AES-CCM authenticated frames with sequence numbers
Radio links	Phone over Wi-Fi with variable latency	ESP-NOW low-latency swarm links; 5 GHz on the C5
Protocols	CRTP only; forked cfclient hangs (open issues)	Keep CRTP for cflib/Crazyswarm2; add MAVLink and micro-ROS later
Updates	No OTA	ESP-IDF OTA, secure boot, flash encryption
Testing	No SIL / HIL / unit tests	Host build, simulator bridge, fuzzing, HIL in CI
Docs / support	Some 404 pages; limited support since Dec 2022	Maintained docs as part of this project

3. System architecture



Why this split

- **ESP32-P4:** large compute headroom over the S2/S3, a low-power safety monitor (same-chip; validate actuator shutdown path), and a camera interface plus hardware H.264 encoder for the surveillance variant. The LP core is NOT an independent safety barrier.
- **ESP32-C5 module:** dual-band Wi-Fi 6 (5 GHz is less crowded indoors) and ESP-NOW for drone-to-drone messages. Radio interrupts never touch the flight loop. A pre-certified module can reduce FCC certification work only within its grant and integration constraints; host compliance obligations remain.
- **Fills an upstream gap:** ESP-Drone currently runs the P4 in a no-radio diagnostic profile and notes it still needs a companion radio design.
- **Failure isolation:** if the C5 or Wi-Fi fails, the P4 keeps stabilizing and runs its failsafe (hover, then land); the ELRS link remains available as a manual override.

Firmware structure

Layer	Contents	Runs on
fc_core (portable C library)	Filters, estimator (EKF), rate/attitude/position control, mixer, commander, safety supervisor. No FreeRTOS calls, no dynamic allocation.	P4 and PC (SITL / unit tests)
HAL interface	Time, sensors, motors, link, storage. One implementation per target (P4 board, simulator).	P4, PC
Drivers	ICM-42688-P, BMP581, PMW3901, VL53L1X, INA226, DShot (RMT), CRSF, SPI link	P4
Link firmware	SPI frame bridge, ESP-NOW swarm, Wi-Fi UDP (CRTP), time sync, provisioning	C5
Tools	Swarm dongle firmware, ground scripts, Crazyswarm2 config, web configurator (later)	C5 dongle, PC

4. Indoor swarm drone (Rev A build)

4.1 Airframe

Option	Verdict
Brushed micro (ESP-Drone / Crazyflie style)	Cheap, but brushed motors wear unevenly so the four drones drift apart in behavior; little payload.
2-inch ducted brushless whoop (SELECTED)	Ducts protect props and people in frequent swarm crashes; cheap spares in US stores (GetFPV, RaceDayQuads, BetaFPV, NewBeeDrone); payload for flow, ToF and markers.
3-inch+ open props	Too dangerous and loud for indoor swarm work.

Part	Selection	Notes
Frame	2-inch (85-90 mm) ducted, 20x20 mm FC mount (or printed adapter)	Custom 3D-printed TPU frame later
Motors	1103-1104 brushless	40 mm props
ESC	Off-the-shelf 4-in-1, Bluejay firmware	Bidirectional DShot; replaceable separately
Battery	1S HV 450-550 mAh (2S optional)	Choose after weight check in simulation
Weight target	50-70 g all-up	Well under 250 g
Markers	3-4 IR reflective markers, unique pattern per drone	For mocap identification

4.2 Flight controller sensors and parts

Function	Part	Reason
IMU	ICM-42688-P (SPI)	Low noise, 8 kHz, low cost; footprint for a second IMU
Barometer	BMP581	Low noise altitude reference
Flow / height	PMW3901 (or PAA3905) + VL53L1X	Indoor hover; drivers exist in ESP-Drone
Power monitor	INA226 + shunt	Accurate battery state for failsafes
Storage	16 MB flash + microSD	Blackbox logging
Radio	ESP32-C5 certified module	Wi-Fi 6 dual-band, ESP-NOW
Optional	UWB (DW3000) footprint, LIS3MDL compass, MIPI-CSI connector	Future swarm self-localization and surveillance

4.3 Indoor positioning

Layer	What	Approx. cost	Role
Onboard	Optical flow + ToF	~$15 / drone	Smooth local hover; backup during mocap dropouts
External (first)	Low-cost mocap: 4x PS3 Eye cameras + IR markers	~$60-100 total	One shared coordinate frame for all drones; ground truth for accuracy measurement
Later	UWB tags + anchors (DW3000)	~$25 / drone + anchors	Decentralized self-localization without cameras

Data path: mocap -> ROS 2 -> Crazyswarm2 -> ESP-NOW swarm dongle -> drones. Onboard, the EKF fuses mocap position, flow, ToF and IMU so a drone keeps flying through short tracking losses.

5. Control and communication

Stage	Control method	Approx. cost	Why
Simulation and bench	Laptop + gamepad over Wi-Fi	$0	Easiest; works with existing Crazyflie tools
Swarm	Laptop + USB ESP32-C5 swarm dongle (ESP-NOW)	~$8	One radio reaches all drones with low latency
Every real flight	ELRS handheld radio as kill switch / manual override	~$40-60	Works even if Wi-Fi or the C5 fails
Demos	Phone app	$0	Nice to have, not primary

Link protocol requirements

- P4 <-> C5: framed SPI protocol with length, type, sequence number and CRC-16; separate priority lanes for commands, telemetry and bulk data (video later).
- Air links: CRTP carried over ESP-NOW and UDP; every command frame authenticated (AES-CCM) with a monotonically increasing sequence number to block replay.
- Swarm status broadcast at 50 Hz: ID, position, velocity, battery, flight state, timestamp.
- Time sync from the dongle so all drones share a common clock for coordinated maneuvers.
- Failsafe timing: no valid command for 0.5 s -> hold position; 2 s -> controlled landing (values to be tuned in simulation).

6. Verification gates (test before you build)

No purchase or assembly happens until the previous gate passes. Pass criteria are initial targets and may be refined after the G1 model is built.

Gate	What is verified	Tools	Pass criteria (proposed)
G1 Math	Rigid-body model, control loop margins, sensor noise impact, estimator behavior	Python: numpy, scipy, python-control	Phase margin > 45 deg; stable over +/-20% mass and thrust variation
G2 SIL	Real flight C code compiled for PC, flying in simulation; 4-drone swarm	gym-pybullet-drones, Gazebo + CrazySim, Crazyswarm2	Hover within +/-5 cm; figure-8 error < 10 cm; 4 drones with zero collisions
G3 Code	Warnings as errors, static analysis, unit tests, sanitizers, parser fuzzing, AI code review	clang-tidy, cppcheck, CTest, ASan/UBSan, libFuzzer	Zero warnings; > 80% coverage of control code; 1 h fuzzing without crashes
G4 PCB	ERC/DRC, scripted netlist checks (strapping pins, pull-ups, power budget), power-stage simulation, DFM, AI schematic review	KiCad, ngspice, Python	No brownout in simulation at full motor current spikes; zero DRC errors
G5 HIL	Real P4 running firmware with simulated sensor data; motors disconnected	USB bridge to simulator	Flight-loop jitter < 5 us; 4 kHz loop holds with Wi-Fi and logging active
G6 Flight	Props off -> tethered -> single hover -> 2 drones -> 4 drones	Blackbox logs + mocap ground truth	Real logs match simulation within agreed tolerance

7. Roadmap

Phase	Scope	Gate(s)	Approx. cost	Approx. time
1 - Software foundation	Repo, portable flight core, PC build, math model, SITL, 4-drone swarm simulation, protocol specs	G1-G3	$0	4-6 weeks
2 - Dev-board bring-up	P4 dev board + sensor breakouts; drivers, P4<->C5/C6 link, safety watchdog, HIL	G5	~$120-180	3-5 weeks
3 - Custom FC board	KiCad P4 + C5 design, review, order 5 assembled boards (4 drones + spare)	G4	~$25-40 / board (small volume)	4-6 weeks
4 - Swarm flight	Build 4 drones, mocap setup, ESP-NOW swarm, Crazyswarm2 missions, optional UWB	G6	Frames, motors, ESCs, batteries, mocap	4-8 weeks
5 - Surveillance variant	Camera, H.264 over 5 GHz, GPS, outdoor failsafes, Remote ID	G1-G6 repeated	TBD	TBD

8. Phase 1 task list (software only)

Goal: a 4-drone swarm flying in simulation on the real flight code, passing gates G1-G3, before any hardware is bought.

#	Task	Output	Done when
1.1	Repo + toolchain: monorepo (firmware/ sim/ hardware/ tools/ ci/ docs/), fork esp-drone master (post-PR #137), Docker with pinned ESP-IDF 5.5, GitHub Actions building P4 + C5 firmware	Working repo + CI	CI passes on every push
1.2	Separate flight code from hardware: move estimator, controller, mixer, filters, commander and supervisor into fc_core behind a HAL	fc_core library	Builds for ESP32 and PC
1.3	PC build + tests: CMake host target, unit test framework, ASan/UBSan, coverage	Test harness	Tests run in CI
1.4	G1 math model: 6-DOF dynamics with frame mass/inertia, motor lag, thrust curve; datasheet noise models; stability margins; Monte Carlo	sim/model + report	Phase margin > 45 deg; robust to +/-20%
1.5	G2 SIL: synchronous fc_step() loop (sensor calibration + LPF -> Kalman -> commander watchdog -> controller -> mixer) shared by SITL and the P4; in-process ctypes lockstep with the validated 6-DOF model (one library copy per drone for swarms); firmware PARAM/LOG access by name; Gym-style environment for RL; pybullet kept for obstacles/camera rendering later	SITL bridge + RL env	Single drone hovers within +/-5 cm
1.6	Flight-core upgrades: 4 kHz rate loop, 500 Hz-1 kHz estimator, dynamic notch + RPM hook, newer arming supervisor, mocap + flow EKF fusion	Updated core	Figure-8 error < 10 cm in simulation
1.7	Link protocols: P4<->C5 SPI framing, CRTP over ESP-NOW, AES-CCM + sequence numbers, 50 Hz swarm broadcast	docs/protocols.md + codec library	1 h fuzzing without crashes
1.8	4-drone swarm simulation with Crazyswarm2: takeoff, formation, position swap, landing; inject link loss, low battery, mocap dropout	Swarm sim scripts	All missions complete; every fault ends in a safe landing
1.9	G3 quality gate: clang-tidy, cppcheck, warnings as errors, AI review checklist, timing budget	CI report	Zero warnings; > 80% control-code coverage

9. AI and autonomy track

Learned components output setpoints (position/velocity/attitude targets), never raw motor commands; the verified PID loop and safety supervisor remain the fallback. Training runs on a workstation, with simulation validation under the same gates. On-board inference only after validation.

What fits on the ESP32-P4

Task	Model	Reported P4 performance	Fit
Person / object detection (surveillance)	ESPDet-Pico, 224x224 input (Espressif esp-detection)	About 51 ms inference, over 18 FPS incl. pre/post-processing	Good: detect-and-follow at low speed
General multi-class detection	YOLO11n, 640x640	About 2.75 s per frame	Too slow for flight; use for offline/ground processing
RL flight/swarm policy	Small MLP (e.g. 2 x 64 neurons)	Not benchmarked; depends on network size, quantization and concurrent load	Candidate for high-level setpoints; benchmark before deployment

Detection figures are published Espressif / community benchmarks, not yet measured on this hardware; the flight loop occupies one P4 core, so CNN frame rate will be measured in Phase 2.

Roadmap

Phase	AI work
Phase 1 (now)	SITL exposes a Gym-style environment (reset/step) so policies can be trained on a workstation
Phase 4 (swarm)	Workstation-trained RL for formation keeping and collision avoidance in the 4-drone simulation. On-board MLP inference from ESP-NOW neighbour states only after validation and benchmarking
Phase 5 (surveillance)	On-board person detection on the P4 camera input (MIPI-CSI) with ESP-DL; detect-and-follow behaviour gated by the safety supervisor

Safety rules for learned components

- Outputs are bounded setpoints only.
- The supervisor can always override (geofence, attitude/rate limits, battery).
- Every policy must pass the same SIL/HIL gates as hand-written code.
- Policy inputs and outputs are logged in the blackbox.

10. Phase 2 shopping list (buy only after G3 passes)

Item	Qty	Purpose
ESP32-P4-Function-EV-Board (includes ESP32-C6)	1	Two-chip development before the custom board
ICM-42688-P breakout	1	IMU driver and timing tests
BMP581 breakout (e.g., SparkFun)	1	Barometer driver
PMW3901 + VL53L1X breakouts	1 each	Flow and height drivers
INA226 breakout	1	Power monitor driver
ESP32-C5 DevKit	1	Swarm USB dongle firmware
PS3 Eye cameras	2 (4 later)	Early mocap testing
Logic analyzer (8 ch, low cost)	1	Measure loop timing, SPI and DShot signals

Estimated total: about $120-180 before the custom board.

11. US regulatory notes

Topic	Notes
Indoor flight	FAA airspace rules generally do not apply to flights fully indoors; no registration or Remote ID needed for indoor-only research.
Radio (FCC)	Use an FCC-certified ESP32-C5 module (not the bare chip) within its grant and integration constraints (antenna, placement, RF exposure and labeling). Module certification is not an exemption from all FCC host testing; host emissions, RF exposure and other applicable compliance obligations remain. The P4 has no radio. Verify certification and integration requirements for the selected ELRS gear.
Outdoor (later)	Under 250 g recreational: registration not required unless flying under Part 107. Over 250 g or research under Part 107: registration and Remote ID required (the C5 can broadcast it).
Surveillance	State privacy and recording laws apply. Fly only in spaces you control during research.

This is general guidance, not legal advice; verify current FAA/FCC rules before outdoor flight.

12. Risks and mitigations

Risk	Impact	Mitigation
ESP32-P4 silicon revision / ESP-IDF maturity	Driver gaps or errata	Bring up on the official dev board first; pin ESP-IDF version; track errata
P4 <-> C5 link complexity	Schedule slip	Spec and fuzz the protocol in Phase 1; fallback to ESP-Hosted for Wi-Fi-only operation
Weight budget on a 2-inch frame	Short flight time	Weight model in G1; 2S option; minimize optional parts on swarm build
Mocap occlusion / reflections	Position dropouts	EKF fusion with flow + ToF; hold-then-land failsafe; matte room surfaces
2.4 GHz interference indoors	Link latency	Use 5 GHz on the C5; ESP-NOW channel planning; ELRS for safety link
Higher power draw of a two-chip design	Battery life	Measure in Phase 2; power down unused peripherals
GPL-3.0 obligations	Must publish source of distributed firmware	Accepted; keep all changes in a public repo
Crashes during swarm testing	Hardware damage, injury	Ducted frames, ELRS kill switch, netted test area, simulation first
Learned policies behaving unexpectedly outside training conditions	Unsafe commands	Setpoint-only interface, supervisor override, domain randomization in simulation, SIL/HIL gates before flight

13. Immediate next steps

- Create the monorepo and fork ESP-Drone master into firmware/ (Task 1.1).
- Set up the Docker toolchain with ESP-IDF 5.5 and CI builds for ESP32-P4 and ESP32-C5.
- Start the Python 6-DOF model using measured specs of the selected 2-inch frame (Task 1.4).
- Draft docs/protocols.md for the P4 <-> C5 link and swarm messages (Task 1.7).